Local / On-Premise Server Deployment Guide

Face-Verified Attendance App Backend (NestJS + Microsoft SQL Server)

This guide provides step-by-step instructions to deploy, configure, and maintain the **Face-Verified Attendance App Backend** on an **in-house local server** or **office machine** (Windows Server/PC or Ubuntu Linux) using **Microsoft SQL Server (MSSQL)**.

Table of Contents

- Deployment Architectures (Choose Your Setup)
- Server Hardware & Software Prerequisites
- Step 1: Assign a Static Local IP to the Server
- Step 2: Install Node.js, Git & PM2
- Step 3: Microsoft SQL Server (MSSQL) Database Setup
- Step 4: Deploy & Configure the Application
- Step 5: Process Management & Auto-Start with PM2
- Step 6: Firewall & Port Configuration
- Step 7: Optional Reverse Proxy & Domain / SSL (Nginx / Cloudflare Tunnel)
- Step 8: Configuring the Android App (APK)
- Step 9: Database Backups & Maintenance

---

1. Deployment Architectures (Choose Your Setup)

Depending on your office networking, choose one of the following setups:

```
+-------------------------------------------------------------------------------+
| OPTION A: Office Wi-Fi / Local Intranet Only (Zero Public Exposure)           |
| Phones on Office Wi-Fi ---> Local Server IP (e.g. http://192.168.1.100:3000)  |
+-------------------------------------------------------------------------------+

+-------------------------------------------------------------------------------+
| OPTION B: Cloudflare Tunnel (Recommended for Public Access + Free SSL)        |
| Phones anywhere ---> https://attendance.yourcompany.com ---> Local Server (Port 3000) |
+-------------------------------------------------------------------------------+

+-------------------------------------------------------------------------------+
| OPTION C: Router Port Forwarding with Static Public IP                        |
| Phones anywhere ---> http://your-public-ip:3000 ---> Router NAT ---> Local Server |
+-------------------------------------------------------------------------------+
```

- **Option A (Local LAN Only)**: Best for security and 100% offline offices. Employees must be connected to office Wi-Fi to punch in / sync history.
- **Option B (Cloudflare Tunnel)**: Best for remote sync with zero router port forwarding and automatic HTTPS SSL.

---

2. Server Hardware & Software Prerequisites

Hardware Requirements

- **CPU**: Dual-Core 2.0 GHz or higher (Intel Core i3/i5/i7, AMD Ryzen, or Xeon).
- **RAM**: 8 GB recommended (4 GB minimum) for running Node.js and SQL Server together.
- **Disk Storage**: 30 GB free SSD storage.
- **Network**: Wired Ethernet connection to your office router (recommended for 24/7 uptime).

Supported Operating Systems

- **Windows (Native MSSQL)**: Windows 10, Windows 11, or Windows Server 2019/2022/2025.
- **Linux (MSSQL on Linux / Docker)**: Ubuntu 22.04 LTS or 24.04 LTS Server / Desktop.

---

3. Step 1: Assign a Static Local IP to the Server

Your server machine must have a permanent IP address on your local network (e.g., `192.168.1.100` or `10.0.0.50`).

On Windows:

Open **Settings** > **Network & Internet** > **Ethernet**.

Click **Edit** next to IP assignment.

Select **Manual** > **IPv4** > Toggle **ON**.

Set IP: `192.168.1.100`, Subnet: `255.255.255.0`, Gateway: `192.168.1.1`, DNS: `8.8.8.8`.

On Ubuntu Linux:

Set a static configuration in netplan:

```
sudo nano /etc/netplan/01-netcfg.yaml
```

```
network:
  version: 2
  ethernets:
    eth0:
      dhcp4: no
      addresses: [192.168.1.100/24]
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

Apply netplan:

```
sudo netplan apply
```

---

4. Step 2: Install Node.js, Git & PM2

On Windows:

Download and install **Node.js 20 LTS** from nodejs.org.

Open PowerShell as Administrator and install PM2:

```
npm install -g pm2
npm install -g pm2-windows-startup
pm2-startup install
```

On Ubuntu Linux:

```
# 1. Update system packages
sudo apt update && sudo apt upgrade -y

# 2. Install Node.js 20 LTS via NodeSource
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs git build-essential

# 3. Install PM2 globally
sudo npm install -g pm2
```

---

5. Step 3: Microsoft SQL Server (MSSQL) Database Setup

On Windows (SQL Server Express or Developer Edition):

Download & Install:

- Download **SQL Server 2022 Express** (Free) from Microsoft.
- Install using **Basic** or **Custom** installation.
- Install **SQL Server Management Studio (SSMS)**.

Enable SQL Server Authentication & TCP/IP (Port 1433):

- Open **SQL Server Configuration Manager**.
- Navigate to **SQL Server Network Configuration** > **Protocols for MSSQLSERVER** (or `SQLEXPRESS`).
- Double-click **TCP/IP** > Set **Enabled = Yes**.
- In the **IP Addresses** tab, scroll to **IPAll** and set **TCP Port = 1433** (clear TCP Dynamic Ports).
- In SSMS, right-click the server instance > **Properties** > **Security** > Select **SQL Server and Windows Authentication mode**.
- Restart the SQL Server service.

Create the Database and User:

Open SSMS or `sqlcmd` and execute:

```
-- 1. Create Database
CREATE DATABASE attendance_prod;
GO

-- 2. Create Login and Database User
CREATE LOGIN attendance_user WITH PASSWORD = 'YourStrongPassword123!', CHECK_POLICY = OFF;
GO

USE attendance_prod;
GO

CREATE USER attendance_user FOR LOGIN attendance_user;
ALTER ROLE db_owner ADD MEMBER attendance_user;
GO
```

On Ubuntu Linux (SQL Server 2022 on Linux):

```
# 1. Import repository GPG keys
curl -fsSL https://packages.microsoft.com/keys/microsoft.asc | sudo gpg --dearmor -o
/usr/share/keyrings/microsoft-prod.gpg
curl -fsSL https://packages.microsoft.com/config/ubuntu/22.04/mssql-server-2022.list | sudo tee
/etc/apt/sources.list.d/mssql-server-2022.list

# 2. Install SQL Server
sudo apt update
sudo apt install -y mssql-server

# 3. Configure SQL Server (choose Developer or Express edition and set sa password)
sudo /opt/mssql/bin/mssql-conf setup

# 4. Verify service status
systemctl status mssql-server --no-pager
```

Create the database via `sqlcmd`:

```
sqlcmd -S localhost -U sa -P 'YourStrongPassword123!' -Q "CREATE DATABASE attendance_prod;"
```

---

6. Step 4: Deploy & Configure the Application

1. Extract the Server Package

Extract your `attendance-server.zip` into your target deployment folder (e.g. `C:\attendance-backend` on Windows or `/var/www/attendance-backend` on Linux).

2. Configure the Production `.env` File

Create `server/.env` with your MSSQL connection:

```
# ============================================================================
# SERVER CONFIGURATION
# ============================================================================
PORT=3000
NODE_ENV=production
BASE_URL=http://192.168.1.100:3000
JWT_SECRET=b79a2f1c84e6012d93e5fa3c907b81da4f6c8d2e1b5a93e7
JWT_EXPIRES_IN=7d

# ============================================================================
# DATABASE CONFIGURATION (Microsoft SQL Server / MSSQL)
# ============================================================================
DB_TYPE=mssql
DB_HOST=localhost
DB_PORT=1433
DB_USERNAME=attendance_user
DB_PASSWORD=YourStrongPassword123!
DB_DATABASE=attendance_prod
# Automatically creates all 6 tables and relationships on first run:
DB_SYNCHRONIZE=true
DB_LOGGING=false
DB_ENCRYPT=false
DB_TRUST_SERVER_CERTIFICATE=true

# ============================================================================
# DEFAULT OFFICE & GEOFENCING CONFIGURATION
# ============================================================================
DEFAULT_OFFICE_ID=DEFAULT_OFFICE
DEFAULT_OFFICE_NAME=Headquarters
DEFAULT_OFFICE_LAT=28.490293
DEFAULT_OFFICE_LNG=77.060544
DEFAULT_OFFICE_RADIUS_M=100
DEFAULT_OFFICE_TIMEZONE=Asia/Kolkata
DEFAULT_MATCH_THRESHOLD=0.55
DEFAULT_SYNC_WINDOWS=["16:00","00:00"]
DEFAULT_SYNC_JITTER_MINUTES=30

# ============================================================================
# STORAGE CONFIGURATION
# ============================================================================
UPLOAD_DIR=uploads
```

3. Install Dependencies & Build Application

```
# 1. Install packages
npm install

# 2. Compile TypeScript
npm run build

# 3. Run seed script (Creates office geofence & default test accounts)
npm run seed
```

---

7. Step 5: Process Management & Auto-Start with PM2

PM2 ensures your backend stays online 24/7, restarts automatically on unexpected crashes, and boots with the system.

Starting the Application:

```
# Start backend server
pm2 start dist/main.js --name "attendance-backend"

# Save running process list for auto-boot
pm2 save

# Setup OS startup script
pm2 startup
```

Useful PM2 Commands:

Command	Description
`pm2 status`	Check server running status, CPU and RAM usage
`pm2 logs attendance-backend`	View live server logs and API requests
`pm2 restart attendance-backend`	Restart the application
`pm2 stop attendance-backend`	Stop the application

---

8. Step 6: Firewall & Port Configuration

Ensure port `3000` is accessible to mobile devices on your local Wi-Fi / network.

On Windows (Windows Defender Firewall):

Open **Windows Defender Firewall with Advanced Security**.

Click **Inbound Rules** > **New Rule...**.

Select **Port** > Click **Next**.

Select **TCP** and enter **Specific local ports**: `3000, 1433`.

Select **Allow the connection** > Check Domain, Private, Public > Name it `Attendance Backend 3000`.

On Ubuntu Linux (UFW Firewall):

```
sudo ufw allow 22/tcp      # SSH management
sudo ufw allow 3000/tcp    # Direct API access
sudo ufw allow 1433/tcp    # SQL Server (if accessed remotely)
sudo ufw allow 80/tcp      # HTTP (if using Nginx)
sudo ufw allow 443/tcp     # HTTPS (if using Nginx/SSL)
sudo ufw enable
```

---

9. Step 7: Optional Reverse Proxy & Domain / SSL (Nginx / Cloudflare Tunnel)

Option A: Using Nginx on Port 80 (Standard HTTP)

```
server {
    listen 80;
    server_name 192.168.1.100;

    client_max_body_size 50M;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Option B: Cloudflare Tunnel (Remote Access + Free Automatic SSL)

If you want employees to sync attendance outside the office without opening router ports:

In Cloudflare Dashboard > **Zero Trust** > **Networks** > **Tunnels** > **Create Tunnel**.

Route your domain (e.g. `https://attendance.yourcompany.com`) to `http://localhost:3000`.

Install and run `cloudflared` service on your server.

---

10. Step 8: Configuring the Android App (APK)

Before compiling the APK for distribution to employees, set your server URL in `android/local.properties`:

```
## Set to your local server IP (or Cloudflare Tunnel domain):
SERVER_BASE_URL=http://192.168.1.100:3000/v1/

## Or if using Cloudflare Tunnel:
# SERVER_BASE_URL=https://attendance.yourcompany.com/v1/
```

Build the APK:

```
cd android
./gradlew assembleRelease
```

Distribute the compiled APK from:

```
android/app/build/outputs/apk/release/app-release.apk
```

---

11. Step 9: Database Backups & Maintenance

Automated Daily MSSQL Backups (PowerShell / Windows Task Scheduler):

Create a script at `C:\attendance-backend\backup_mssql.ps1`:

```
$backupDir = "C:\backups\mssql"
if (!(Test-Path $backupDir)) { New-Item -ItemType Directory -Path $backupDir }
$date = Get-Date -Format "yyyy-MM-dd_HH-mm-ss"
$backupFile = "$backupDir\attendance_prod_$date.bak"

Invoke-Sqlcmd -ServerInstance "localhost" -Username "attendance_user" -Password
"YourStrongPassword123!" -Query "BACKUP DATABASE attendance_prod TO DISK = '$backupFile' WITH
FORMAT, MEDIANAME = 'AttendanceBackups', NAME = 'Full Backup of attendance_prod';"

# Keep backups for 30 days
Get-ChildItem -Path $backupDir -Filter *.bak | Where-Object { $_.CreationTime -lt (Get-
Date).AddDays(-30) } | Remove-Item
```

On Linux (Cron with sqlcmd):

```
#!/bin/bash
BACKUP_DIR="/var/backups/mssql"
mkdir -p $BACKUP_DIR
DATE=$(date +'%Y-%m-%d_%H-%M-%S')
sqlcmd -S localhost -U sa -P 'YourStrongPassword123!' -Q "BACKUP DATABASE attendance_prod TO DISK
= '$BACKUP_DIR/attendance_prod_$DATE.bak' WITH FORMAT;"

# Keep 30 days of backups
find $BACKUP_DIR -type f -name "*.bak" -mtime +30 -exec rm {} \;
```

🎯 Verification Checklist

- [ ] Microsoft SQL Server is running and listening on Port `1433`.
- [ ] Database `attendance_prod` created with user permissions.
- [ ] `DB_TYPE=mssql` and credentials set in `server/.env`.
- [ ] Application compiles with `npm run build` without errors.
- [ ] Seed script ran (`npm run seed`) creating office location and accounts.
- [ ] PM2 shows `online` status (`pm2 status`).
- [ ] Testing endpoint responds: `curl http://192.168.1.100:3000/v1/health` returns `{"status":"ok"}`.
- [ ] Android APK installed and able to login with `EMP1001` / `Password@123`.